AI AUTOMATION PLATFORM

Product Recommendation Bot Complete Flow

Point a bot at any web store's URL and it learns the catalog, then answers shopping questions with grounded, product-specific recommendations — the platform's first end-to-end bot type, built on the knowledge-base ingestion pipeline.

Internal reference · Backend implementation · ai-automation-platform

DESCRIPTION

What this is

This is the platform's first working bot: a **product recommendation assistant** for ecommerce stores. A business gives it one input — a store URL — and gets a chatbot that can answer real shopping questions ("comfy running shoes under $100") by recommending actual products from that store, not generic or invented ones.

It's built from two pieces the platform already models as first-class concepts: a **knowledge base** (the ingested, searchable version of the store's catalog) and a **bot** (a configured AI agent that draws on one or more knowledge bases to answer questions). Connecting a store to a knowledge base, and a knowledge base to a bot, is the whole setup — no manual data entry, no product spreadsheets to upload.

Store URL in | Auto-detected catalog | Grounded recommendations | Chat playground

Behind the scenes, the store's products are automatically discovered, read, and converted into a searchable form (embeddings) the bot can retrieve from at answer time — so every reply is grounded in what the store actually sells, not guessed.

COMPLETE USER FLOW

Store URL → Knowledge Base → Bot → Playground Chat

ACT 1 Turn a store into a knowledge base

Give it a URL once — it does the reading for you.

1. The user opens **Knowledge Bases** and clicks **New Knowledge Base**, entering a name and the store's URL (e.g. `https://example.myshopify.com`).
2. The new knowledge base appears immediately in the list, badged **"Syncing…"** — the user isn't stuck waiting on the request.
3. In the background, the system reads the store: it tries the store's own structured product feed first, falls back to reading embedded product markup on the page, and as a last resort has an AI model read the page itself to find products — whichever approach works for that particular store.
4. Every product found (title, description, price, image, link) is converted into a searchable form and saved.
5. The knowledge base flips to **"Ready"** with a product count — or to a clear **error state** with a reason, if the store's products genuinely couldn't be found.
6. The user can optionally open the knowledge base to browse exactly which products were pulled in.

> If a store simply isn't in a format the system can read, the user sees a clear error instead of an empty or broken bot later.

ACT 2 Create a bot on top of it

Give the bot a name, a personality, and a catalog to know.

1. The user opens **Bots** and clicks **New Bot**.
2. They give it a name and a system prompt (its instructions/personality — e.g. "You are a friendly shopping assistant for our store"), and attach one or more **ready** knowledge bases.
3. The bot is created and appears in the Bots list. Each bot has a **Playground** button.

ACT 3 Talk to it in the Playground

Ask a real shopping question, get a grounded answer.

1. The user clicks **Playground** on a bot card — a chat panel opens.
2. They type a question, e.g. *"comfy running shoes under $100."*
3. The system finds the products from the bot's knowledge base that actually match the question.
4. It hands those matching products, plus the bot's personality/instructions, plus the question, to the AI model to write a natural-language reply.
5. The chat shows the assistant's written reply alongside the matched product cards — image, title, price, link.
6. Each question is answered fresh — the bot doesn't need memory of earlier questions in this version to give a useful, grounded answer every time.